COMP 3069
Computer Graphics

Lab 8: Texture

Autumn 2024

The University of Nottingham

UNITED KINGDOM · CHINA · MALAYSIA

Contents

- Specify UV texture coordinates
- Set up a VAO correctly
- Write shaders to perform simple texture mapping
- Load bitmap files
- Create and bind a texture object
- Bi-linear filtering
- Use OpenGL Tri-linear filtering
- Generate mipmaps

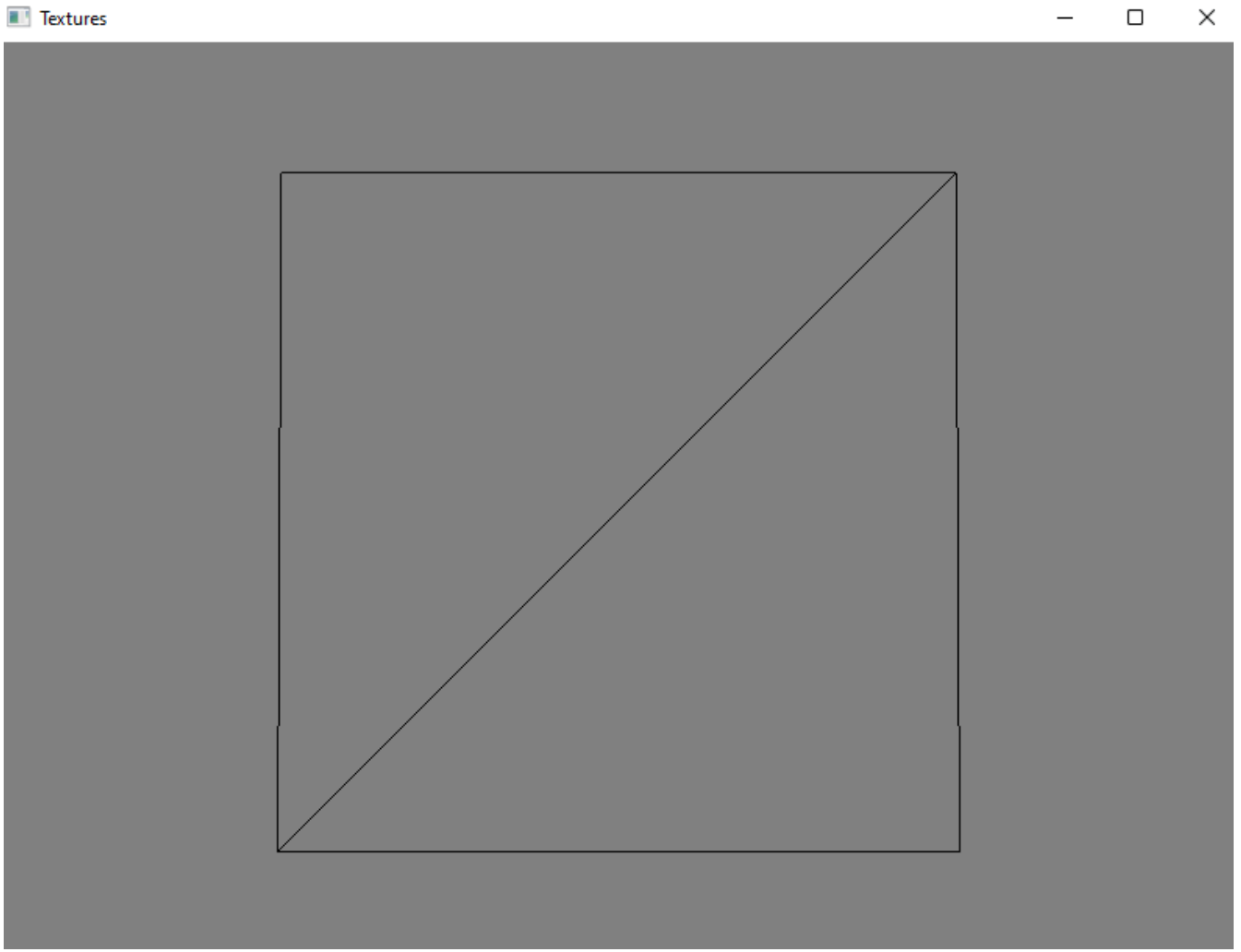
Set-up of Lab8

- Using the instructions in Lab 1, create Lab8 project
- Download `Lab8.cpp` and copy the code to the Lab8.cpp in the new project
- Copy `bitmap.h, camera.h, ModelViewerCamera.h, shader.h, texture.h, util.h,` and `window.h,` and add them to the Header Files
- Copy the vertex shader `texture.vert` and fragment shader `texture.frag`, and add them to the Source Files

Vertex Attributes

```
15      float vertices[] =
16      {
17          //t1
18          //pos                       //tex
19          -0.9f,  0.9f,  0.f,         //tl
20           0.9f,  0.9f,  0.f,         //tr
21           0.9f, -0.9f,  0.f,         //br
22          //t2
23          //pos                       //tex
24          -0.9f,  0.9f,  0.f,         //tl
25           0.9f,  -0.9f,  0.f,        //br
26          -0.9f,   -0.9f,  0.f,       //bl
27      };
```

Default Display



Map Vertices to UVs

```
18      //original task
19      //t1
20      //pos                       //tex
21      -0.9f,  0.9f,  0.f,         0.0f,  1.0f, //tl
22      0.9f,  0.9f,  0.f,          1.0f,  1.0f, //tr
23      0.9f,  -0.9f,  0.f,         1.0f,  0.0f, //br
24      //t2
25      //pos                       //tex
26      -0.9f,  0.9f,  0.f,         0.0f,  1.0f, //tl
27      0.9f,   -0.9f,  0.f,        1.0f,  0.0f, //br
28      -0.9f,   -0.9f,  0.f,       0.0f,  0.0f, //bl
29
```

Load and Use Texture

- In `main`

```
96    GLuint texture = setup_texture("jubilee.bmp");
```

- In `texture.h`

```
6   GLuint setup_texture(const char* filename)
7   {
8       glEnable(GL_TEXTURE_2D);
9       glEnable(GL_BLEND);
10
11      GLuint texObject;
12      glGenTextures(1, &texObject);
13      glBindTexture(GL_TEXTURE_2D, texObject);
14
15      glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
16      glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
17      glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
18      glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);
19
20
21      //glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
22      //glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
23
24      unsigned char* pxls = NULL;
25      BITMAPINFOHEADER info;
26      BITMAPFILEHEADER file;
27      loadbitmap(filename, pxls, &info, &file);
28
29      if (pxls != NULL)
30      {
31          glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, info.biWidth, info.biHeight, 0, GL_RGB, GL_UNSIGNED_BYTE, pxls);
32      }
33      glGenerateMipmap(GL_TEXTURE_2D);
34
35      delete[] pxls;
36
37      glDisable(GL_TEXTURE_2D);
38      glDisable(GL_BLEND);
39
40      return texObject;
41      //return 0;
42  }
```

In texture.h

- To enable texture

```
8       glEnable(GL_TEXTURE_2D);
9       glEnable(GL_BLEND);
```

- To generate a texture object

```
11      GLuint texObject;
12      glGenTextures(1, &texObject);
13      glBindTexture(GL_TEXTURE_2D, texObject);
```

- To draw the texture

```
15      glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
16      glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
17      glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
18      glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);
```

- To load the bitmap file

```
24      unsigned char* pxls = NULL;
25      BITMAPINFOHEADER info;
26      BITMAPFILEHEADER file;
27      loadbitmap(filename, pxls, &info, &file);
```

In bitmap.h

- To enable texture

```
9   GLuint loadbitmap(const char* filename, unsigned char * &pixelBuffer, BITMAPINFOHEADER* infoHeader, BITMAPFILEHEADER *fileHeader)
```

- `filename` is the file location of the bitmap to load
- `pixelBuffer` is a pointer to the pixel colours which will be updated by the function
- `infoHeader` and `fileHeader` are windows structs storing information about bitmap files.

More in texture.h

- To call `glTexImage2D()`

```
29      if (pxls != NULL)
30      {
31          glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, info.biWidth, info.biHeight, 0, GL_RGB, GL_UNSIGNED_BYTE, pxls);
32      }
```

  - `target` should be `GL_TEXTURE_2D`, `level` is just set it to 0, `internalformat` is the number of colour components and we want to use `GL_RGB`, `width` and `height` are the dimensions of the image and can be retrieved from the `BITMAPINFOHEADER` struct, `border` must be 0, `format` should be `GL_RGB`, `type` should be `GL_UNSIGNED_BYTE`, and `data` is the colour data we just loaded
- To delete the colour data, disable textures and return the texture object

```
35      delete[] pxls;
36
37      glDisable(GL_TEXTURE_2D);
38      glDisable(GL_BLEND);
39
40      return texObject;
```

Set up VAO

- The position attribute is 3 floats, and the UV attribute is 2 floats

```
134     unsigned int VAO;
135     glGenVertexArrays(1, &VAO);
136     unsigned int VBO;
137     glGenBuffers(1, &VBO);
138     glBindVertexArray(VAO);
139     glBindBuffer(GL_ARRAY_BUFFER, VBO);
140     glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
141
142     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 5 * sizeof(float), (void*)0);
143     glEnableVertexAttribArray(0);
144     glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 5 * sizeof(float), (void*)(3 * sizeof(float)));
145     glEnableVertexAttribArray(1);
146
147     /*
148     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
149     glEnableVertexAttribArray(0);
150     */
151     glBindBuffer(GL_ARRAY_BUFFER, 0);
152     glBindVertexArray(0);
```

Vertex Shader

```
1   #version 330 core
2
3   layout(location = 0) in vec3 aPos;
4   layout(location = 1) in vec3 aTex;
5
6   uniform mat4 model;
7   uniform mat4 view;
8   uniform mat4 projection;
9
0   out vec2 tex;
1
2   void main()
3   {
4       gl_Position = projection * view * model * vec4(aPos, 1.f);
5       tex = aTex.xy;
6   }
```

Fragment Shader

```
1   #version 330 core
2
3   out vec4 fragColour;
4
5   in vec2 tex;
6
7   uniform sampler2D Texture;
8
9   void main()
10  {
11      fragColour = texture(Texture, tex);
12  }
13
```

Back to Lab8.cpp

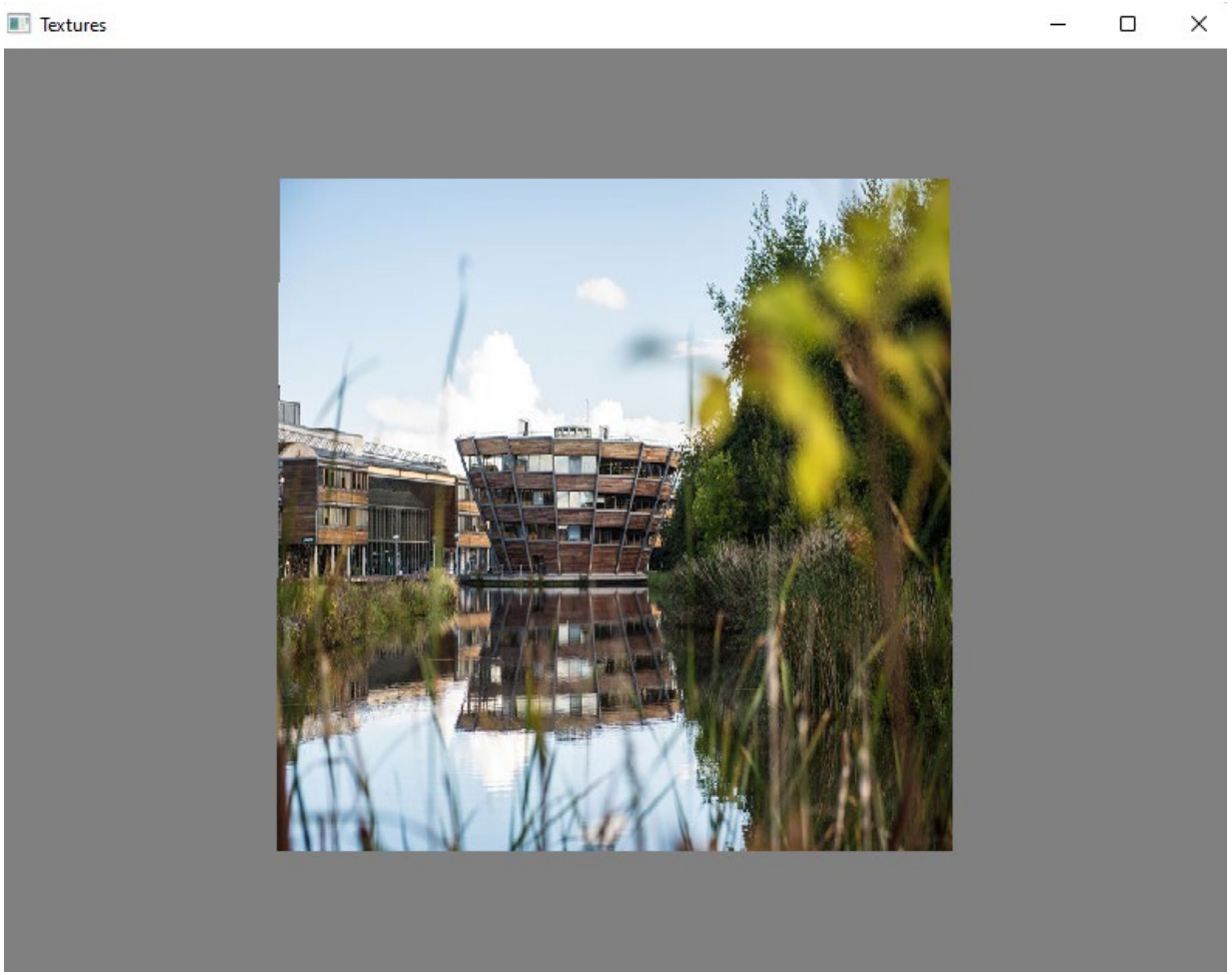
- To change the render type from line to filled

```
163         glClearColor(.5f, .5f, .5f, 1.f);
164         glClear(GL_COLOR_BUFFER_BIT);
165         //glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
166         glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
```

- To bind the texture

```
181         glBindTexture(GL_TEXTURE_2D, texture);
182         glUseProgram(shaderProgram);
183         glBindVertexArray(VAO);
184         glDrawArrays(GL_TRIANGLES, 0, 6);
185         glBindVertexArray(0);
```

Display with Texture

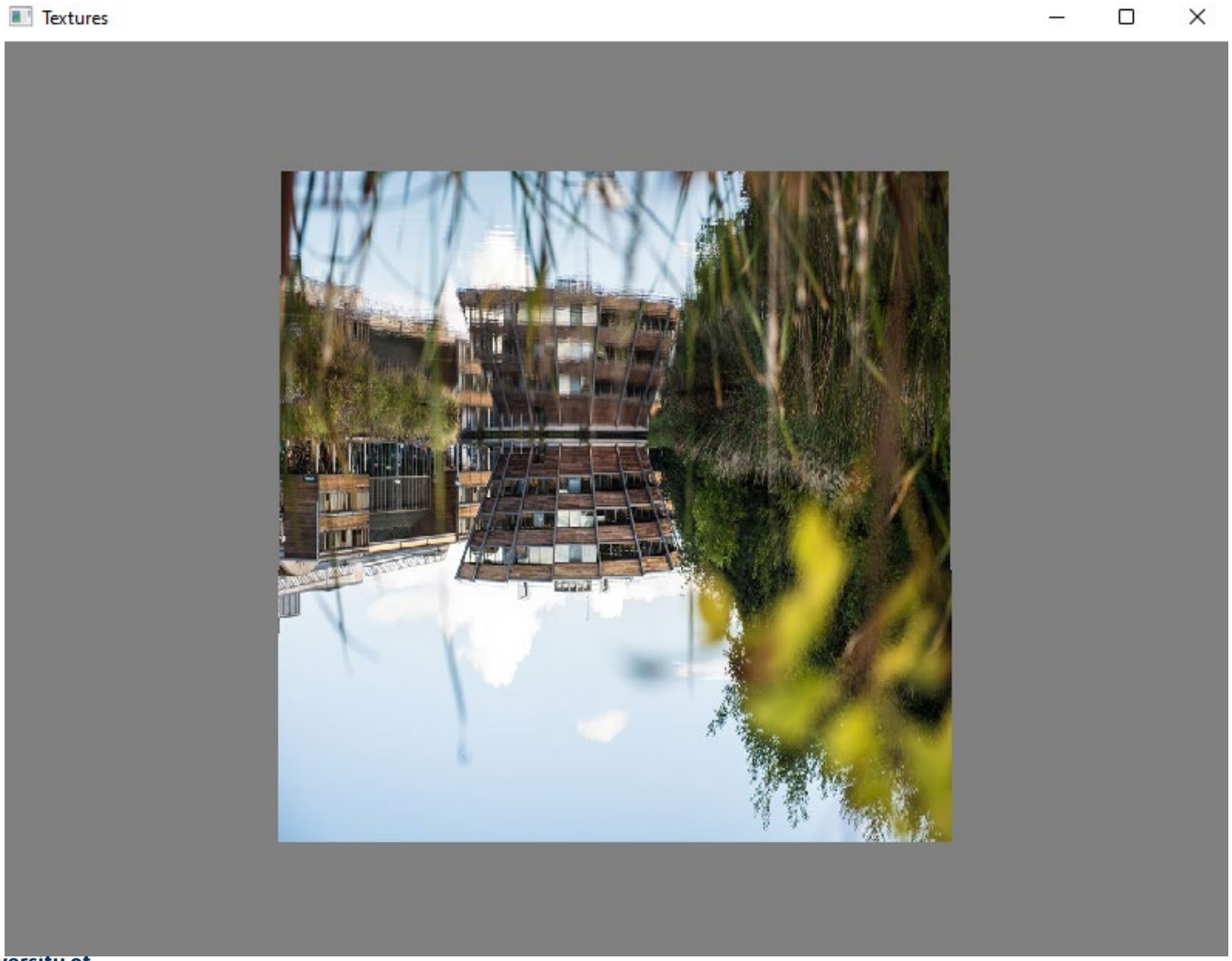


Upside Down

- To change the UV attributes in the vertex array so that the image is mapped upside down like this

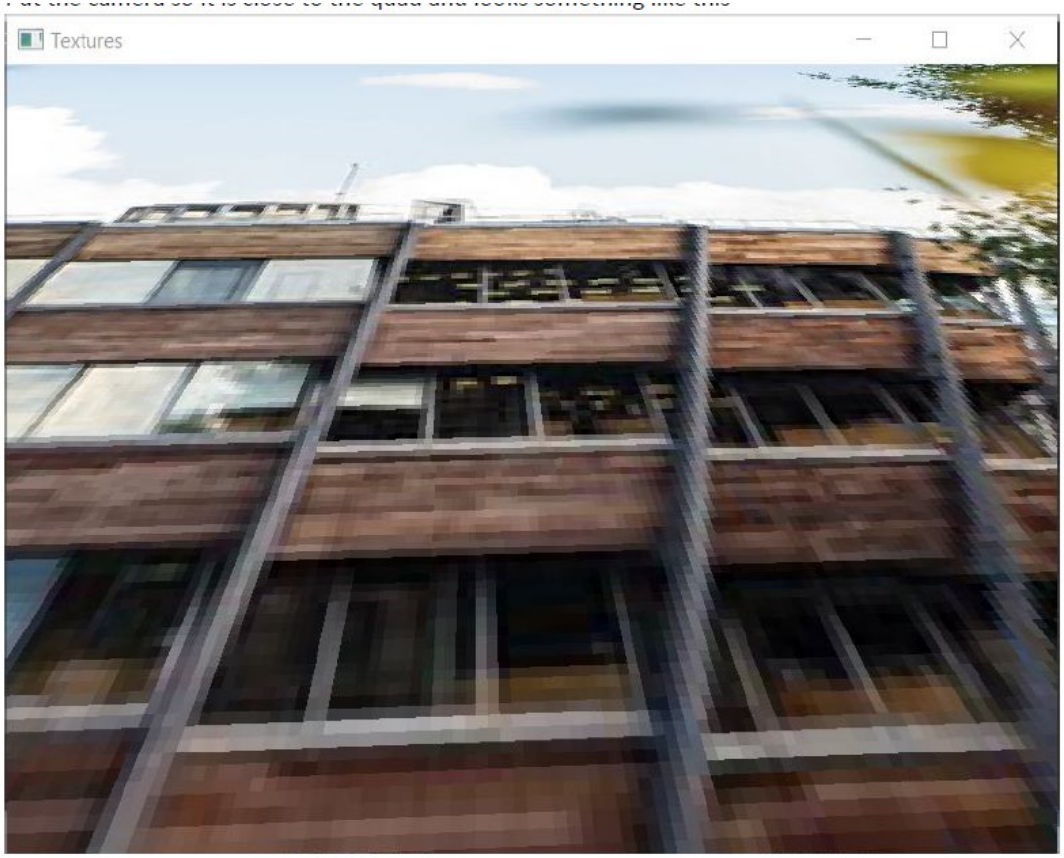
```
32      //additonal task: upside down
33      //t1
34      //pos                   //tex
35      -0.9f, 0.9f, 0.f,       0.0f, 0.0f, //tl
36      0.9f, 0.9f, 0.f,        1.0f, 0.0f, //tr
37      0.9f, -0.9f, 0.f,       1.0f, 1.0f, //br
38      //t2
39      //pos                   //tex
40      -0.9f, 0.9f, 0.f,       0.0f, 0.0f, //tl
41      0.9f,  -0.9f, 0.f,      1.0f, 1.0f, //br
42      -0.9f,  -0.9f, 0.f,     0.0f, 1.0f, //bl
```

Up Side Down Display



Magnification

- The camera is so close to the quad that magnification is happening

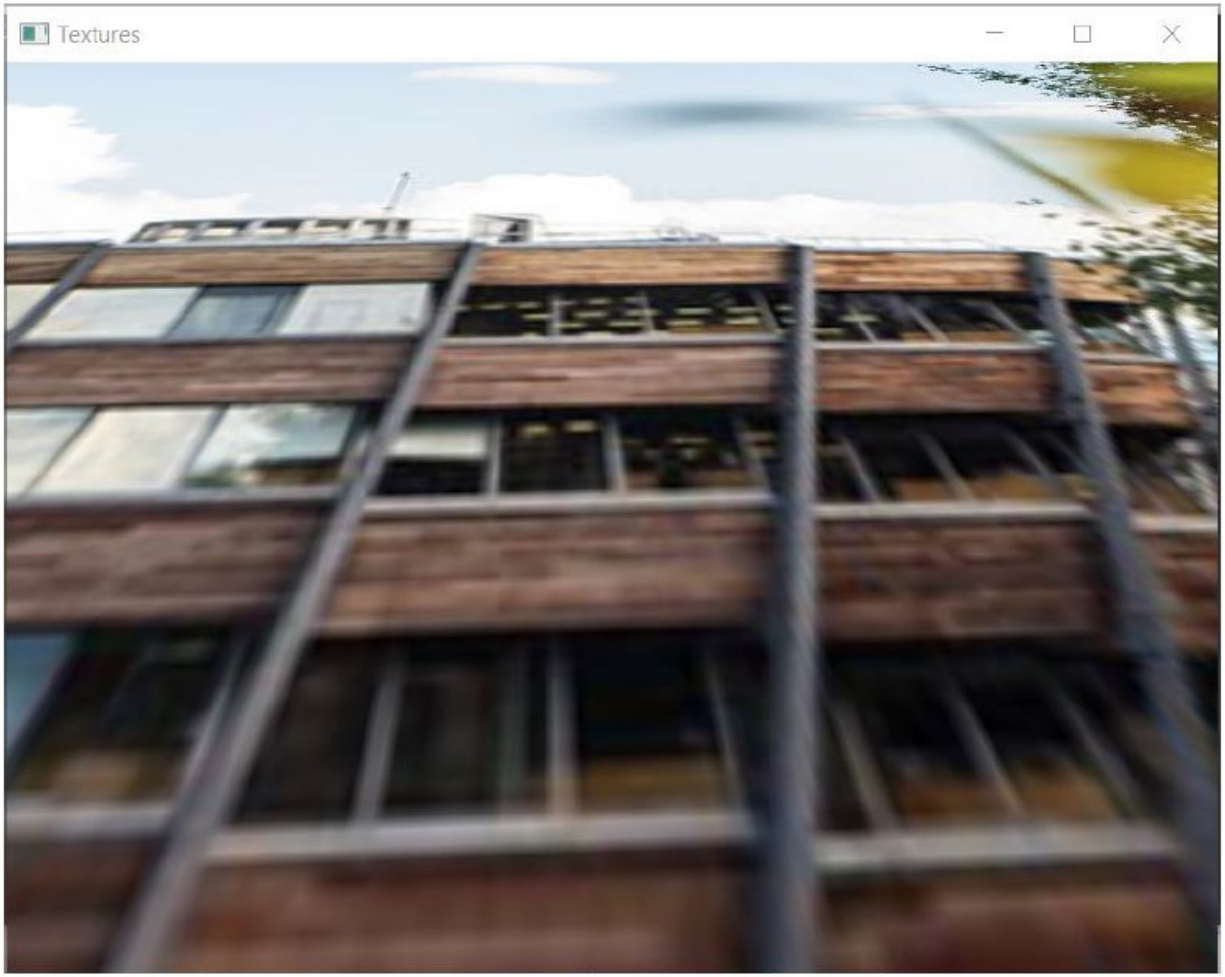


Bi-linear filtering

- In texture.h set the magnification filter so that it uses `GL_LINEAR`

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
```

Smooth Image with Bilinear Filtering



Mipmaps & Trilinear Filtering

```
unsigned char* pxls[16];
BITMAPINFOHEADER info[16];
BITMAPFILEHEADER file[16];
for (int c = 0; c < n; c++)
{
    loadbitmap(filename[c], pxls[c], &info[c], &file[c]);

    if (pxls != NULL)
    {
        glTexImage2D(GL_TEXTURE_2D, c, GL_RGB, info[c].biWidth, info[c].biHeight, 0, GL_RGB, GL_UNSIGNED_BYTE, pxls[c]);
    }

    delete[] pxls[c];
}
```

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);
```

Back to Lab8.cpp

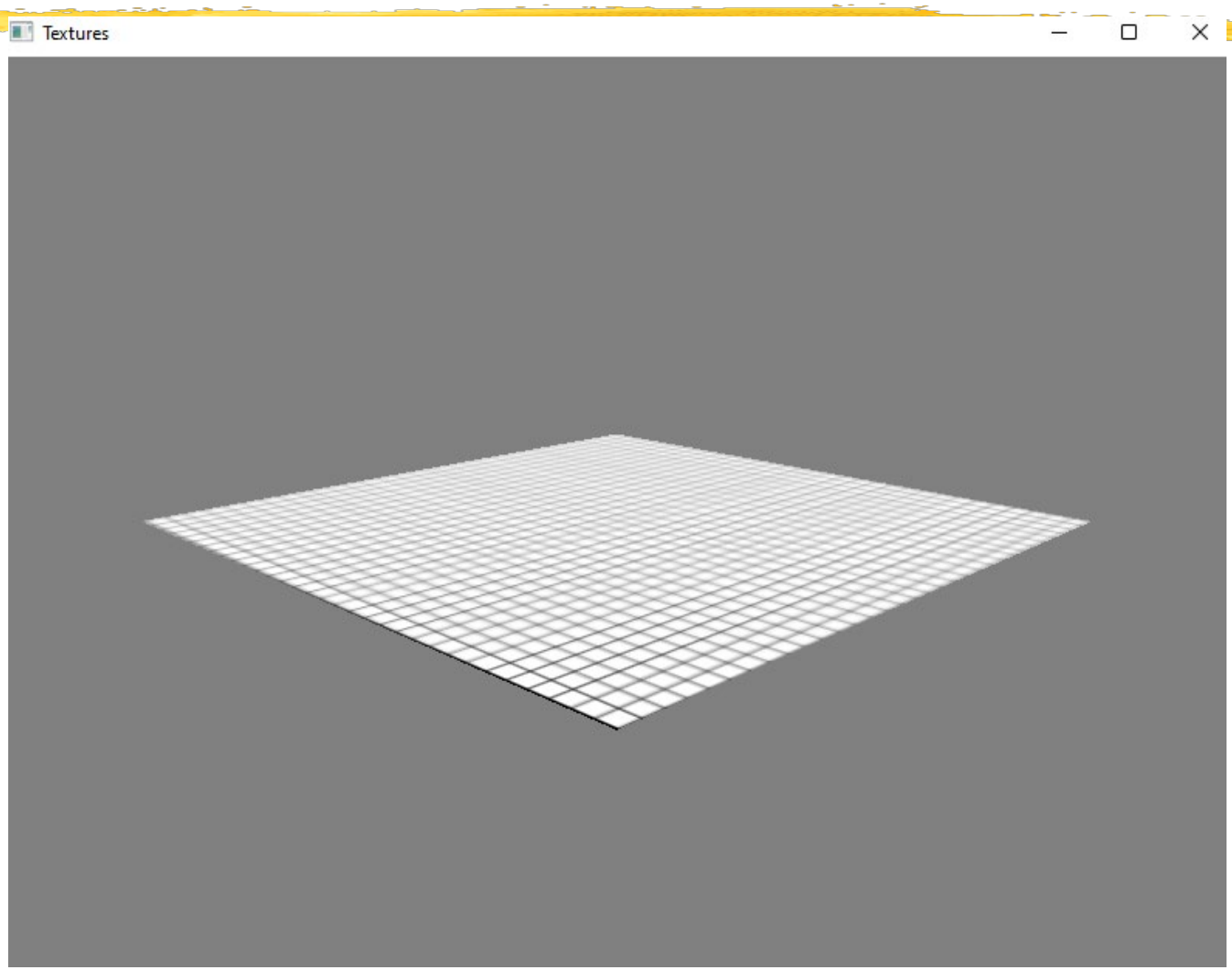
- Set up an array of paths to these 11 files and call the new setup_mipmaps() function

```
115     const char* files[11] = {
116         "mipmaps/grid1024x1024.bmp",
117         "mipmaps/grid512x512.bmp",
118         "mipmaps/grid256x256.bmp",
119         "mipmaps/grid128x128.bmp",
120         "mipmaps/grid64x64.bmp",
121         "mipmaps/grid32x32.bmp",
122         "mipmaps/grid16x16.bmp",
123         "mipmaps/grid8x8.bmp",
124         "mipmaps/grid4x4.bmp",
125         "mipmaps/grid2x2.bmp",
126         "mipmaps/grid1x1.bmp"
127     };
128     GLuint texture = setup_mipmaps(files, 11);
```

- Initialise the camera

```
112     InitCamera(Camera, -45, 15);
113     MoveAndOrientCamera(Camera, glm::vec3(0, 0, 0), cam_dist, 0.f, 0.f);
```

Result Using Mipmaps

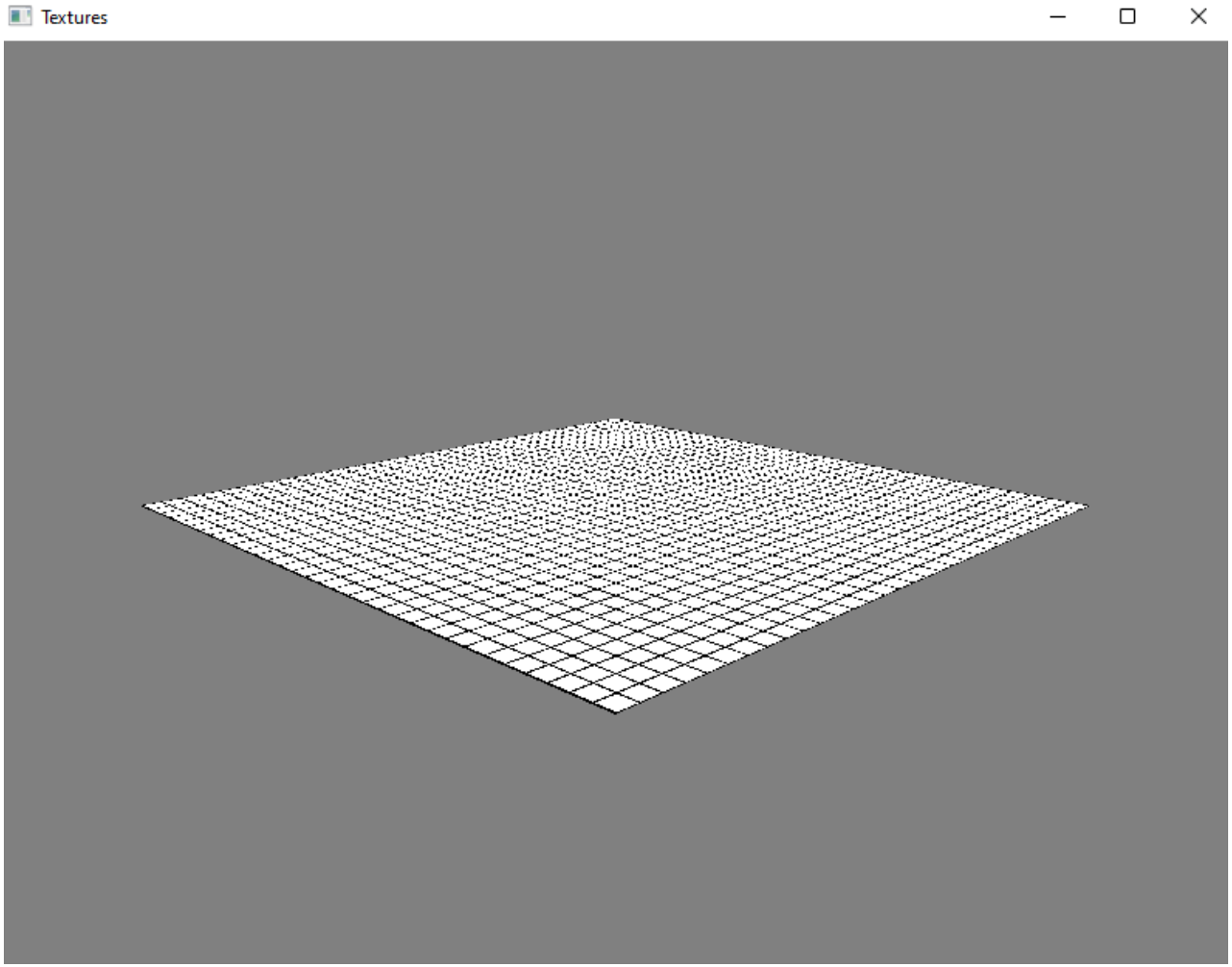


Change to Nearest Approach

- In texture.h in the function

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
```

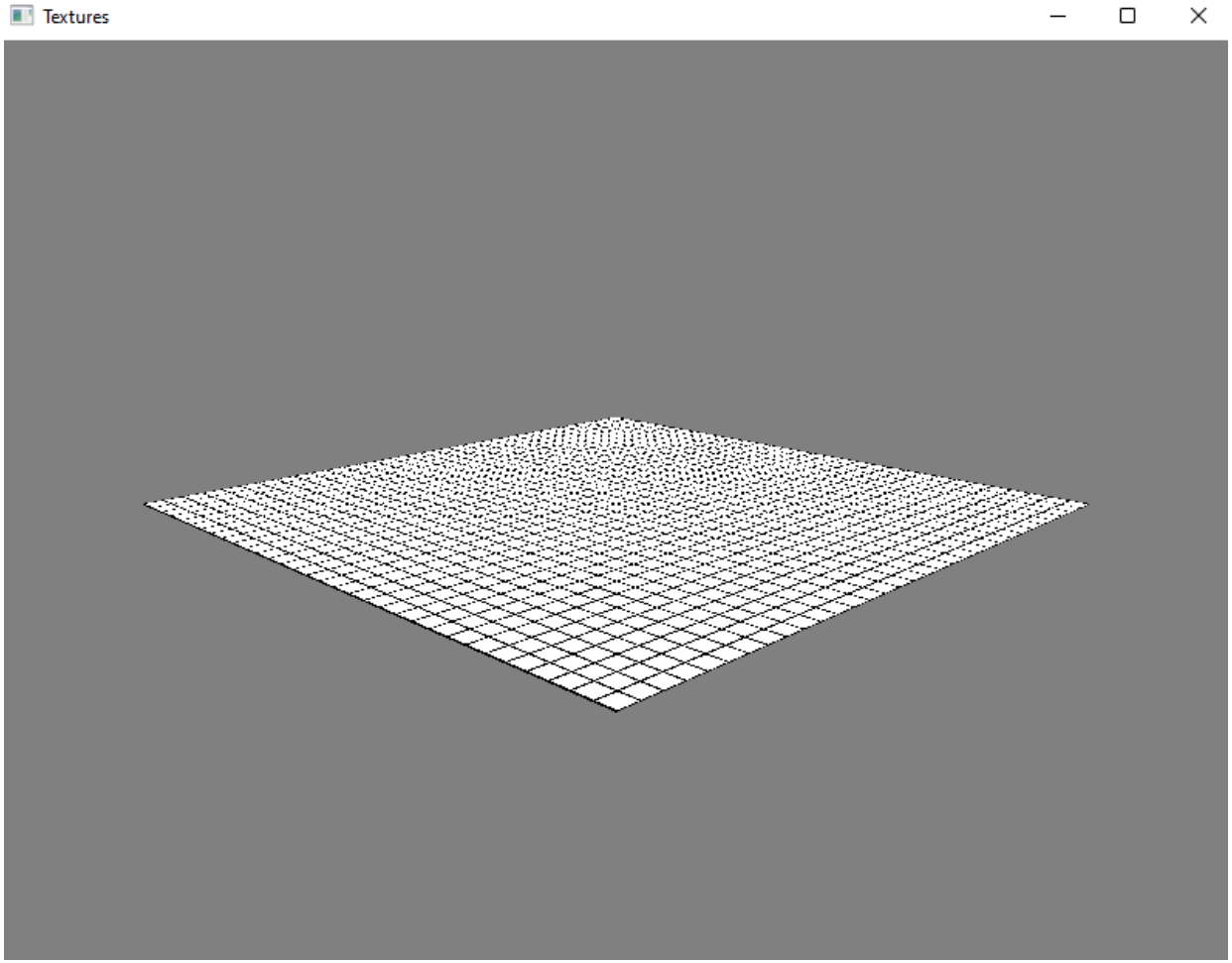
Results with Nearest Approach



Not Using Pre-defined Grids

```
]/*  const char* files[11] = {
        "mipmaps/grid1024x1024.bmp",
        "mipmaps/grid512x512.bmp",
        "mipmaps/grid256x256.bmp",
        "mipmaps/grid128x128.bmp",
        "mipmaps/grid64x64.bmp",
        "mipmaps/grid32x32.bmp",
        "mipmaps/grid16x16.bmp",
        "mipmaps/grid8x8.bmp",
        "mipmaps/grid4x4.bmp",
        "mipmaps/grid2x2.bmp",
        "mipmaps/grid1x1.bmp"
    };
    GLuint texture = setup_mipmaps(files, 11);*/
    GLuint texture = setup_texture("grid.bmp");
```

Results without Using Linear Filtering



Generate Mipmaps and Use Tr-linear Filtering

- Generate mipmaps

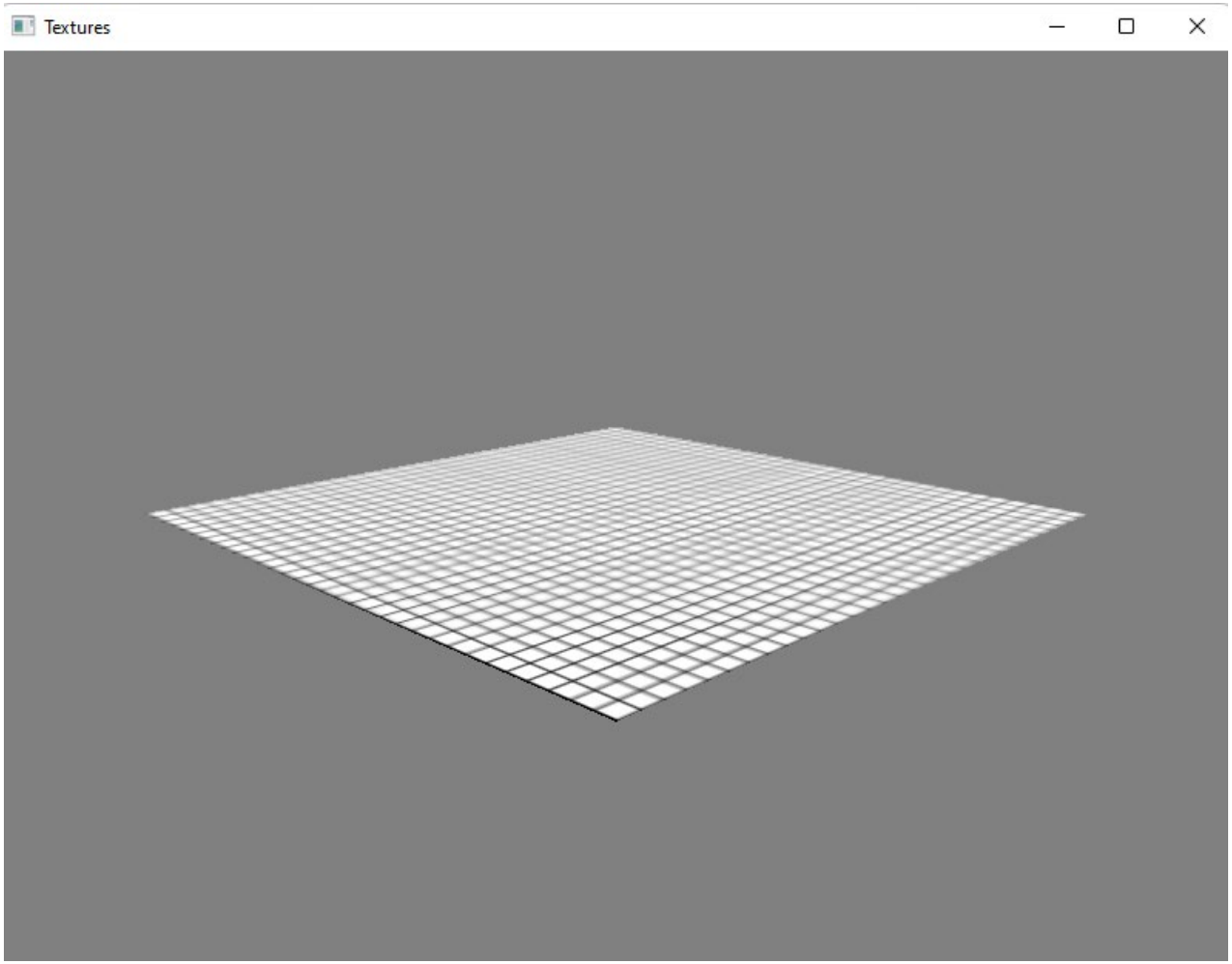
```
if (pxls != NULL)
{
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, info.biWidth, info.biHeight, 0, GL_RGB, GL_UNSIGNED_BYTE, pxls);
}

glGenerateMipmap(GL_TEXTURE_2D);
```

- Set minification to trilinear filtering

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);
```

Results with Tri-linear Filtering



References

- *COMP3069 Lab8 notes*
- *Interactive Computer Graphics: A Top-Down Approach with Shader-Based OpenGL*, **Chapter 7**